Project Information			
Group ID	30		**Semester** *(WS2?)*
1. Student Name	Georgios Sandalidis		1
2. Student Name	Jan Grüttefien		1
3. Student Name	Jenny Dang		2
4. Student Name	Thi Trang Anh Vu		2
Project Title	ThreadChat: A Replicated Threaded Group Chat		

Project Description	ThreadChat is a command-line group chat written in Python. Users pick a username, join channels and chat with everyone in them. Any message can start a thread, and replies appear indented under the message they answer. Users can also react to messages, edit or delete their own, see who is online and view the history. The chat runs on three to five replicated servers on different laptops that find each other automatically. No message is lost when a server crashes, and a reply never appears before the message it answers.

Architectural Model	Why: A single server would be a single point of failure: one crash would lose the history and disconnect every user. How: Hybrid. The servers form a peer group of equal replicas, each with the full chat log. Each client connects to one server and switches to another if it fails. TCP is used for client-server traffic, the election ring and history transfer; UDP for discovery broadcasts, heartbeats and chat messages between servers. Multithreading: one thread per socket and per client, plus a heartbeat timer. No multiprocessing, since the work is network-bound.

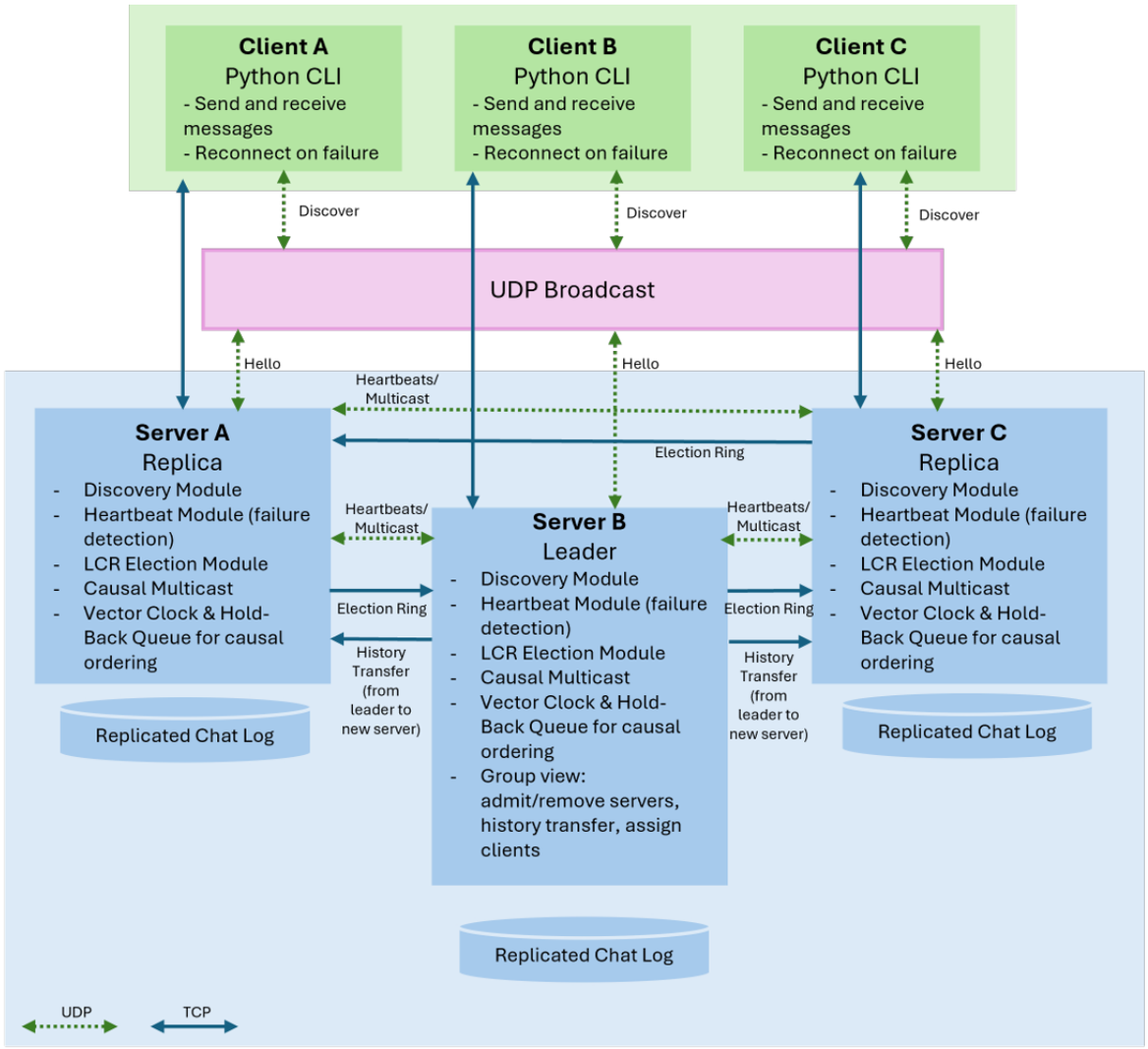
Dynamic Discovery	Why: Servers start, crash and rejoin on different machines at runtime, so no addresses are known in advance. Clients must find a live server at startup and after their server crashes. How: UDP broadcast. A new server broadcasts HELLO; the leader admits it, sends it the chat history and announces it to all servers. If no leader answers, an election starts. A client broadcasts DISCOVER; the leader replies with a server to use and a backup list. Discovery runs at startup and whenever a server or client joins.

Fault Tolerance	Why: Users must keep chatting without losing messages when a server, the leader or a laptop crashes. UDP messages can also get lost. How: We tolerate crash faults (leader, regular server, client) and omission faults. Servers send each other UDP heartbeats every second; three misses count as a failure. Clients ping their server every two seconds. Leader crash: new election. Server crash: the leader removes it from the group. Client crash: its session is removed. If a client's server crashes, the client reconnects to another server and resends unconfirmed requests, which are deduplicated.

Voting	Why: One server must decide group membership; otherwise simultaneous joins or crashes could leave servers with different member lists. The leader admits new servers, sends them the history, removes crashed servers and assigns clients. Chat messages bypass the leader, so chat continues during an election. How: LeLann-Chang-Roberts on a ring ordered by UUID; the highest UUID wins. UUIDs stay unique even with several servers per machine. Elections run at startup, when a server joins and when the leader fails; crashed servers in the ring are skipped.

Reliable Ordered Multicast	Why: Every server must forward its clients' messages to all others so everyone sees everything. A reply often comes from a user on a different server than the question, so FIFO is not enough. Total order is unnecessary, since unrelated messages sent at the same time may appear in any order. How: Causal ordering with vector clocks: a message is held back until everything its sender had seen has been delivered. Reliability: per-sender sequence numbers; a gap triggers a NACK, and any server can resend the message because all keep the full log.

System Architecture Design
Include a clear diagram of your system architecture (key components and connections between them).



Inside each server:

